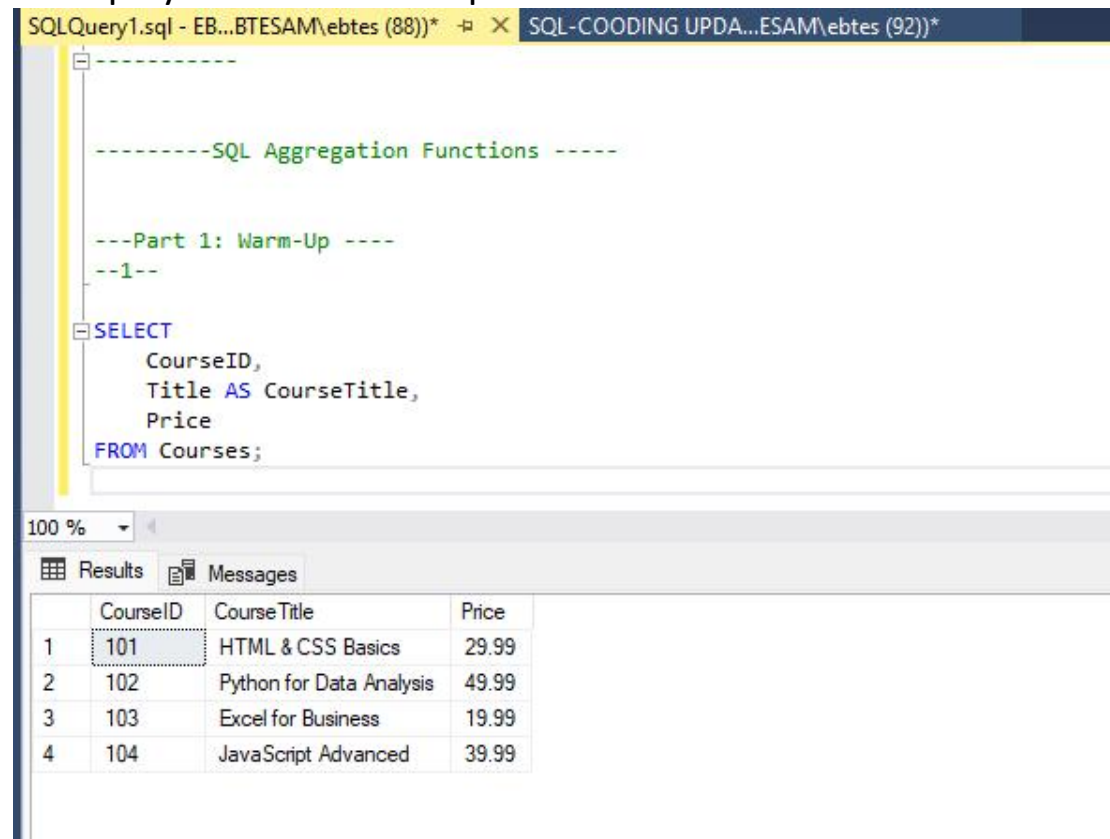


Core Aggregation Practice

Part 1: Warm-Up

1. Display all courses with prices.



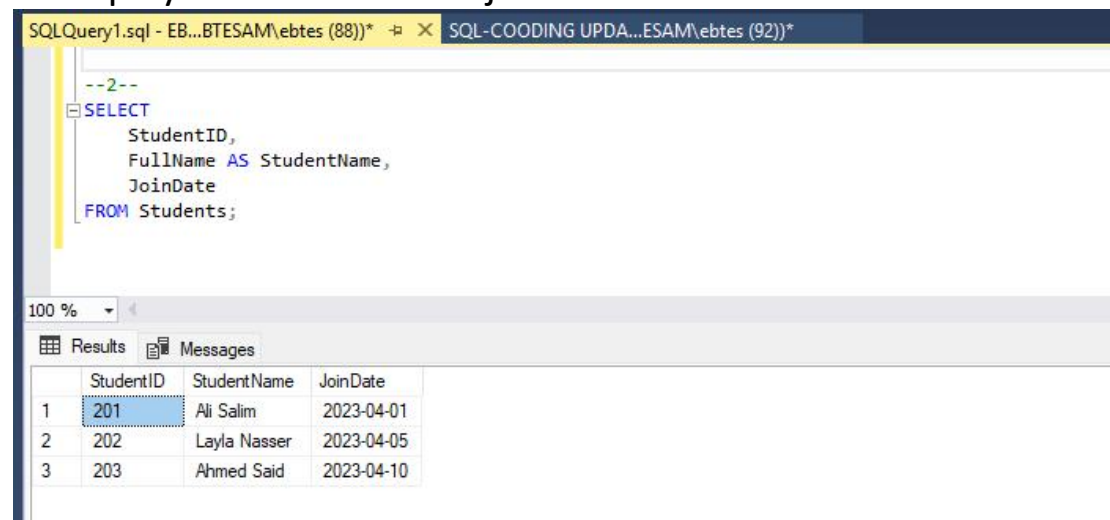
The screenshot shows a SQL query window with the following text:

```
-----SQL Aggregation Functions -----  
  
---Part 1: Warm-Up ---  
--1--  
  
SELECT  
    CourseID,  
    Title AS CourseTitle,  
    Price  
FROM Courses;
```

Below the query window, the 'Results' tab is active, displaying a table with 4 rows and 4 columns:

	CourseID	CourseTitle	Price
1	101	HTML & CSS Basics	29.99
2	102	Python for Data Analysis	49.99
3	103	Excel for Business	19.99
4	104	JavaScript Advanced	39.99

2. Display all students with join dates.



The screenshot shows a SQL query window with the following text:

```
--2--  
  
SELECT  
    StudentID,  
    FullName AS StudentName,  
    JoinDate  
FROM Students;
```

Below the query window, the 'Results' tab is active, displaying a table with 3 rows and 4 columns:

	StudentID	StudentName	JoinDate
1	201	Ali Salim	2023-04-01
2	202	Layla Nasser	2023-04-05
3	203	Ahmed Said	2023-04-10

3. Show all enrollments with completion percent and rating.

SQLQuery1.sql - EB...BTESAM\ebtes (88))* X SQL-COODING UPDA...ESAM\ebtes (92))*

```
--3--  
SELECT  
    EnrollmentID,  
    StudentID,  
    CourseID,  
    EnrollDate,  
    CompletionPercent,  
    Rating  
FROM Enrollments;
```

100 %

Results Messages

	EnrollmentID	StudentID	CourseID	EnrollDate	CompletionPercent	Rating
1	1	201	101	2023-04-10	100	5
2	2	202	102	2023-04-15	80	4
3	3	203	101	2023-04-20	90	4
4	4	201	102	2023-04-22	50	3
5	5	202	103	2023-04-25	70	4
6	6	203	104	2023-04-28	30	2
7	7	201	104	2023-05-01	60	3

3. Count instructors who joined in 2023.

SQLQuery1.sql - EB...BTESAM\ebtes (88))* X SQL-COODING UPDA...ESAM\ebtes (92))*

```
--4--  
SELECT COUNT(*) AS InstructorsJoined2023  
FROM Instructors  
WHERE YEAR(JoinDate) = 2023;
```

100 %

Results Messages

	InstructorsJoined2023
1	2

5. Count students who joined in April 2023.

SQLQuery1.sql - EB...BTESAM\ebtes (88))* X SQL-COODING UPDA...ESAM\ebtes (92))*

```
--5--  
SELECT COUNT(*) AS StudentsJoinedApril2023  
FROM Students  
WHERE YEAR(JoinDate) = 2023  
AND MONTH(JoinDate) = 4;
```

100 %

Results Messages

	StudentsJoinedApril2023
1	3

Part 2: Beginner Aggregation

1. Count total number of students.

The screenshot shows a SQL query window with the following query:

```
SELECT COUNT(*) AS TotalStudents
FROM Students;
```

The query results are displayed in a table with one row:

TotalStudents
3

2. Count total number of enrollments.

The screenshot shows a SQL query window with the following query:

```
SELECT COUNT(*) AS TotalEnrollments
FROM Enrollments;
```

The query results are displayed in a table with one row:

TotalEnrollments
7

3. Find average rating per course.

The screenshot shows a SQL query window with the following query:

```
SELECT
    C.Title AS CourseTitle,
    AVG(E.Rating) AS AvgRating
FROM Courses C
JOIN Enrollments E
    ON C.CourseID = E.CourseID
GROUP BY C.Title;
```

The query results are displayed in a table with four rows:

	CourseTitle	AvgRating
1	Excel for Business	4
2	HTML & CSS Basics	4
3	JavaScript Advanced	2
4	Python for Data Analysis	3

4. Count courses per instructor.

SQLQuery1.sql - EB...BTESAM\ebtes (88))* SQL-COODING UPDA...ESAM\ebtes (92))*

```
SELECT
    I.FullName AS InstructorName,
    COUNT(C.CourseID) AS TotalCourses
FROM Instructors I
LEFT JOIN Courses C
    ON I.InstructorID = C.InstructorID
GROUP BY I.FullName;
```

100 %

Results Messages

	InstructorName	TotalCourses
1	Mohammed Al-Busaidi	2
2	Sarah Ahmed	2

5. Count courses per category.

SQLQuery1.sql - EB...BTESAM\ebtes (88))* SQL-COODING UPDA...ESAM\ebtes (92))*

```
---5---
SELECT
    Cat.CategoryName,
    COUNT(C.CourseID) AS TotalCourses
FROM Categories Cat
LEFT JOIN Courses C
    ON Cat.CategoryID = C.CategoryID
GROUP BY Cat.CategoryName;
```

100 %

Results Messages

	CategoryName	TotalCourses
1	Business	1
2	Data Science	1
3	Web Development	2

6. Count students enrolled in each course.

SQLQuery1.sql - EB...BTESAM\ebtes (88))* SQL-COODING UPDA...ESAM\ebtes (92))*

```
---6---  
  
SELECT  
    C.Title AS CourseTitle,  
    COUNT(E.StudentID) AS TotalStudents  
FROM Courses C  
LEFT JOIN Enrollments E  
    ON C.CourseID = E.CourseID  
GROUP BY C.Title;
```

100 %

Results Messages

	Course Title	TotalStudents
1	Excel for Business	1
2	HTML & CSS Basics	2
3	JavaScript Advanced	2
4	Python for Data Analysis	2

7. Find average course price per category.

SQLQuery1.sql - EB...BTESAM\ebtes (88))* SQL-COODING UPDA...ESAM\ebtes (92))*

```
---7---  
  
SELECT  
    Cat.CategoryName,  
    AVG(C.Price) AS AvgCoursePrice  
FROM Categories Cat  
LEFT JOIN Courses C  
    ON Cat.CategoryID = C.CategoryID  
GROUP BY Cat.CategoryName;
```

100 %

Results Messages

	CategoryName	AvgCoursePrice
1	Business	19.990000
2	Data Science	49.990000
3	Web Development	34.990000

8. Find maximum course price.

SQLQuery1.sql - EB...BTESAM\ebtes (88))* X SQL-COODING UPDA...ESAM\ebtes (92))*

```
--8---
```

```
SELECT MAX(Price) AS MaxCoursePrice
FROM Courses;
```

100 %

Results Messages

	MaxCoursePrice
1	49.99

9. Find min, max, and average rating per course.

SQLQuery1.sql - EB...BTESAM\ebtes (88))* X SQL-COODING UPDA...ESAM\ebtes (92))*

```
--9---
```

```
SELECT
  C.Title AS CourseTitle,
  MIN(E.Rating) AS MinRating,
  MAX(E.Rating) AS MaxRating,
  AVG(E.Rating) AS AvgRating
FROM Courses C
JOIN Enrollments E
  ON C.CourseID = E.CourseID
GROUP BY C.Title;
```

100 %

Results Messages

	Course Title	MinRating	MaxRating	AvgRating
1	Excel for Business	4	4	4
2	HTML & CSS Basics	4	5	4
3	JavaScript Advanced	2	3	2
4	Python for Data Analysis	3	4	3

10. Count how many students gave rating = 5

SQLQuery1.sql - EB...BTESAM\ebtes (88))* X SQL-COODING UPDA...ESAM\ebtes (92))*

```
--10---
```

```
SELECT COUNT(*) AS StudentsWithRating5
FROM Enrollments
WHERE Rating = 5;
```

100 %

Results Messages

	StudentsWithRating5
1	1

Part 3: Extended Beginner Practice

11. Count enrollments per month.

SQLQuery1.sql - EB...BTESAM\ebtes (88))* X SQL-COODING UPDA...ESAM\ebtes (92))*

```
--1--  
SELECT  
    YEAR(EnrollDate) AS EnrollYear,  
    MONTH(EnrollDate) AS EnrollMonth,  
    COUNT(*) AS TotalEnrollments  
FROM Enrollments  
GROUP BY YEAR(EnrollDate), MONTH(EnrollDate)  
ORDER BY EnrollYear, EnrollMonth;
```

100 %

Results Messages

	EnrollYear	EnrollMonth	TotalEnrollments
1	2023	4	6
2	2023	5	1

12. Find average course price overall.

SQLQuery1.sql - EB...BTESAM\ebtes (88))* X SQL-COODING UPDA...ESAM\ebtes (92))*

```
--2--  
SELECT AVG(Price) AS AvgCoursePrice  
FROM Courses;
```

100 %

Results Messages

	AvgCoursePrice
1	34.990000

13. Count students per join month.

SQLQuery1.sql - EB...BTESAM\ebtes (88))* X SQL-COODING UPDA...ESAM\ebtes (92))*

```
--3--  
SELECT  
    YEAR(JoinDate) AS JoinYear,  
    MONTH(JoinDate) AS JoinMonth,  
    COUNT(*) AS TotalStudents  
FROM Students  
GROUP BY YEAR(JoinDate), MONTH(JoinDate)  
ORDER BY JoinYear, JoinMonth;
```

100 %

Results Messages

	JoinYear	JoinMonth	TotalStudents
1	2023	4	3

14. Count ratings per value (1–5).

SQLQuery1.sql - EB...BTESAM\ebtes (88))* SQL-COODING UPDA...ESAM\ebtes (92))*

```
--4--  
  
SELECT  
    Rating,  
    COUNT(*) AS CountPerRating  
FROM Enrollments  
GROUP BY Rating  
ORDER BY Rating;
```

100 %

Results Messages

	Rating	CountPerRating
1	2	1
2	3	2
3	4	3
4	5	1

15. Find courses that never received rating = 5.

SQLQuery1.sql - EB...BTESAM\ebtes (88))* SQL-COODING UPDA...ESAM\ebtes (92))*

```
--5--  
  
SELECT  
    C.CourseID,  
    C.Title AS CourseTitle  
FROM Courses C  
LEFT JOIN Enrollments E  
    ON C.CourseID = E.CourseID  
GROUP BY C.CourseID, C.Title  
HAVING SUM(CASE WHEN E.Rating = 5 THEN 1 ELSE 0 END) = 0;
```

100 %

Results Messages

	CourseID	CourseTitle
1	102	Python for Data Analysis
2	103	Excel for Business
3	104	JavaScript Advanced

16. Count courses priced above 30.

SQLQuery1.sql - EB...BTESAM\ebtes (88))* SQL-COODING UPDA...ESAM\ebtes (92))*

```
---6---  
SELECT COUNT(*) AS CoursesAbove30  
FROM Courses  
WHERE Price > 30;
```

100 %

Results Messages

	CoursesAbove30
1	2

17. Find average completion percentage.

SQLQuery1.sql - EB...BTESAM\ebtes (88))* SQL-COODING UPDA...ESAM\ebtes (92))*

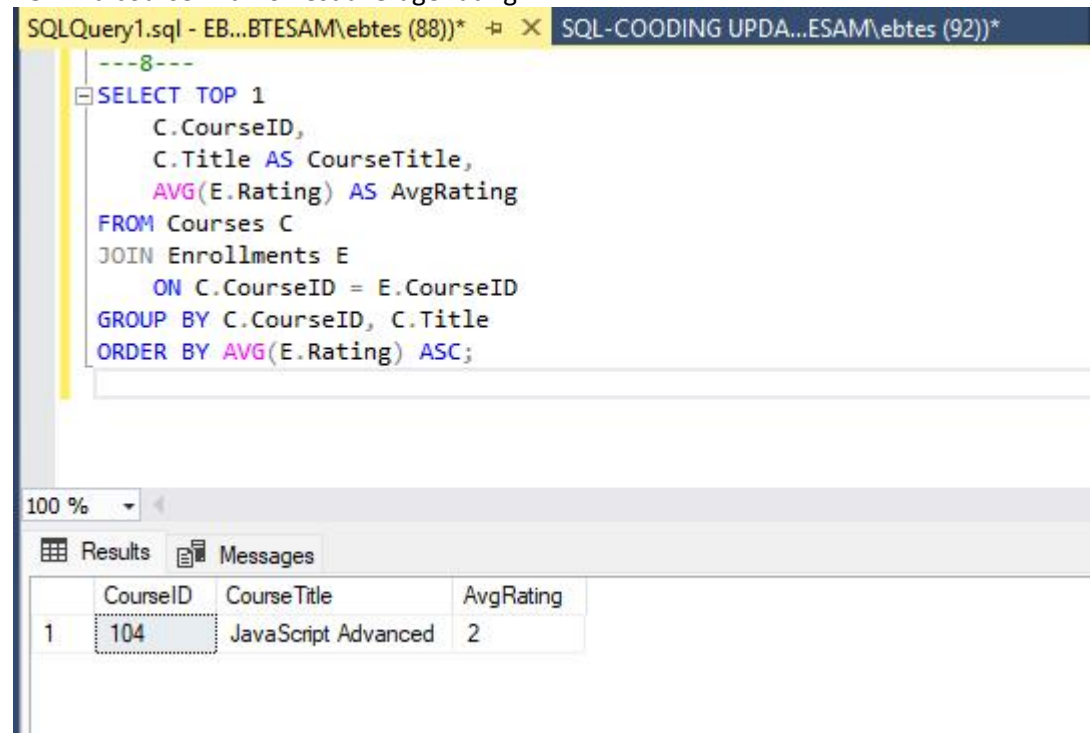
```
---7---  
SELECT AVG(CompletionPercent) AS AvgCompletionPercent  
FROM Enrollments;
```

100 %

Results Messages

	AvgCompletionPercent
1	68

18. Find course with lowest average rating



The screenshot shows a SQL query window with the following text:

```
---8---
SELECT TOP 1
    C.CourseID,
    C.Title AS CourseTitle,
    AVG(E.Rating) AS AvgRating
FROM Courses C
JOIN Enrollments E
    ON C.CourseID = E.CourseID
GROUP BY C.CourseID, C.Title
ORDER BY AVG(E.Rating) ASC;
```

Below the query window, the 'Results' tab is active, displaying a table with the following data:

	CourseID	CourseTitle	AvgRating
1	104	JavaScript Advanced	2

Reflection (End of Day 1)

Answer briefly:

- What was easiest? Aggregation
- What was hardest? join
- What does GROUP BY do in your own words?
used to group rows that have the same values in specified columns into summary rows